

Contents

1 TrashUQ: Plataforma Edge AI con Aprendizaje Federado para Monitorización Inteligente de Residuos	1
1.1 Abstract	2
1.2 Executive Summary	2
1.3 1. Introducción	3
1.4 2. System Overview	3
1.5 3. Hardware and Edge Layer	4
1.6 4. MQTT Communication Layer	6
1.7 5. Backend Architecture	8
1.8 6. Database Layer	9
1.9 7. Frontend Dashboard	10
1.10 8. Federated Learning Layer	10
1.11 9. Edge Simulator and No-Hardware Demo	12
1.12 10. Real Model Runtime Preparation	12
1.13 11. Verification and Testing	13
1.13.1 Start full stack	13
1.13.2 Backend health	13
1.13.3 Frontend proxy	13
1.13.4 MQTT monitor	13
1.13.5 Edge simulator	13
1.13.6 gRPC smoke	13
1.13.7 Model load	14
1.13.8 DB verification	14
1.14 12. Results	14
1.15 13. Limitaciones	15
1.16 14. Future Work	15
1.17 15. Demo Script	15
1.18 16. Deployment Topology	16
1.19 17. Glossary	17
1.20 18. Conclusión	17

1 TrashUQ: Plataforma Edge AI con Aprendizaje Federado para Monitorización Inteligente de Residuos

Subtítulo: Edge AI + Federated Learning trash detection dashboard

Proyecto: TrashUQ

Repositorios: TrashUQ/ y edge/

Equipo / autores: Equipo TrashUQ

Fecha: 2026-05-14

Versión del informe: 1.0

Resumen ejecutivo. TrashUQ implementa una plataforma edge-

to-cloud para monitorizar dispositivos de clasificación de residuos mediante MQTT, persistencia en PostgreSQL, visualización en un dashboard Next.js y coordinación inicial de Federated Learning vía gRPC. El sistema ya demuestra una ruta real de telemetría desde el cliente edge hasta la base de datos y la interfaz. La inferencia real con cámara está preparada mediante un runtime TFLite, mientras que el entrenamiento federado local y `SubmitUpdate` permanecen como trabajo futuro.

1.1 Abstract

TrashUQ es una plataforma de monitorización inteligente de residuos basada en Edge AI, MQTT y Federated Learning. Los nodos edge, ya sea mediante el simulador sin hardware o mediante el runtime preparado para cámara/modelo real, publican estado, métricas, clasificaciones, eventos, logs y solicitudes de ayuda en tópicos MQTT bajo `arduino/<device-id>/...`. El backend FastAPI se suscribe a `arduino/+/#`, persiste los mensajes en PostgreSQL y expone un estado agregado mediante `/api/dashboard/bootstrap`. El frontend Next.js consume ese estado inicial, escucha actualizaciones en vivo por MQTT Web-Socket y visualiza dispositivos, métricas, historial, clasificaciones y eventos. En paralelo, el backend ejecuta un coordinador gRPC de Federated Learning en `localhost:50051`; actualmente `Join` y `GetGlobalModel` funcionan como smoke test real, mientras que `SubmitUpdate` se omite de forma explícita porque el repositorio edge todavía no produce actualizaciones locales entrenadas.

1.2 Executive Summary

Capability	Estado	Evidencia
MQTT edge telemetry	Implementado	<code>edge/app/edge_simulator.py</code> publica en <code>arduino/<device-id>/...</code> ; <code>TrashUQ/backend/app/mqtt_runtime.py</code> se suscribe a <code>arduino/+/#</code> .
Backend persistence	Implementado	<code>mqtt_messages</code> , <code>device_status_latest</code> , <code>device_status_history</code> y <code>coordinator_metrics</code> en <code>TrashUQ/backend/app/db.py</code> .

Capability	Estado	Evidencia
Dashboard live monitoring	Implementado	<code>/api/dashboard/bootstrap</code> + MQTT WebSocket <code>ws://localhost:9001/mqtt</code> en <code>TrashUQ/frontend/app/page.tsx</code> .
gRPC FL Join/GetGlobalModel	Implementado	<code>TrashUQ/backend/app/fl.proto</code> , <code>grpc_server.py</code> y <code>edge/scripts/test_fl_grpc.py</code> .
Real camera inference	Preparado	<code>edge/app/model_runner.py</code> , <code>camera_runtime.py</code> , <code>real_edge_runtime.py</code> .
SubmitUpdate / local FL training	Pendiente	<code>edge/scripts/test_fl_grpc.py</code> lo omite; no existe payload local real de entrenamiento.

1.3 1. Introducción

La monitorización de residuos urbanos requiere información operacional de baja latencia: estado de los dispositivos, consumo de recursos, eventos de clasificación, confianza del modelo y trazabilidad histórica. En un entorno real, transmitir vídeo sin procesar hacia la nube es costoso, frágil y poco escalable. TrashUQ adopta una arquitectura Edge AI para ejecutar inferencia cerca del sensor y enviar únicamente telemetría estructurada.

MQTT encaja con este tipo de sistema porque es ligero, tolerante a enlaces inestables y está diseñado para comunicación publish/subscribe entre dispositivos. PostgreSQL aporta persistencia verificable e histórico para reconstruir el estado del dashboard después de reinicios. El frontend ofrece una vista operacional para demos y validación. La capa de Federated Learning prepara el camino para entrenar modelos distribuidos sin centralizar datos crudos, aunque el entrenamiento local real aún no está implementado.

1.4 2. System Overview

TrashUQ se divide en dos repositorios:

Repositorio	Responsabilidad
TrashUQ/	Docker Compose, Mosquitto, PostgreSQL, backend FastAPI/gRPC, frontend Next.js.

Repositorio	Responsabilidad
edge/	Cliente MQTT, simulador, smoke tests gRPC, wrapper de modelo TFLite y runtime de cámara/imagen/vídeo.

La arquitectura operativa es una ruta edge-to-cloud real:

1. El dispositivo edge publica telemetría MQTT.
2. Mosquitto recibe mensajes en 1883 y WebSocket en 9001.
3. El backend se suscribe a `arduino/+/#`.
4. El backend persiste cada mensaje en PostgreSQL.
5. `/api/dashboard/bootstrap` reconstruye estado inicial e histórico reciente.
6. El dashboard muestra estado y también escucha MQTT WebSocket para actualizaciones live.
7. El cliente edge puede contactar al coordinador FL gRPC para Join y `GetGlobalModel`.

flowchart LR

```

user[Browser user] -->|HTTP :3000| frontend[Next.js frontend\ntrashuq-frontend]
frontend -->|/api/* proxy\nBACKEND_API_URL=http://backend:4000| backend[FastAPI backend\ntrashuq-backend]
frontend -->|MQTT over WebSocket\nws://localhost:9001/mqtt| mqtt[(Mosquitto broker\n:1883)]

edge[Edge node\nsimulator or real runtime] -->|MQTT arduino/<device-id>/...| mqtt
mqtt -->|subscribe arduino/+/#| backend
backend -->|persist telemetry| db[(PostgreSQL\ntrashuq-db :5432)]
backend -->|dashboard bootstrap\n/api/dashboard/bootstrap| frontend

edge -->|gRPC Join / GetGlobalModel\nlocalhost:50051| fl[FL coordinator\nngRPC service :50051]
backend --> fl
fl -->|round/model state| backend

camera[Camera or image/video source] --> model[TFLite classification model\nmodels/trash_classification_model]
model --> edge

```

1.5 3. Hardware and Edge Layer

La capa edge representa el nodo que en producción ejecutará la lectura de cámara, inferencia local y publicación MQTT. El sistema ya puede ejecutarse sin hardware mediante `edge/app/edge_simulator.py`, y también contiene una preparación para inferencia real:

Elemento	Estado	Archivo
Simulador edge sin hardware	Implementado	edge/app/edge_simulator.py
Cliente MQTT reusable	Implementado	edge/app/mqtt_client.py
Cliente gRPC FL smoke	Implementado	edge/app/fl_client.py
Wrapper de modelo real	Preparado	edge/app/model_runner.py
Cámara/imagen/vídeo runtime	Preparado	edge/app/camera_runtime.py, edge/app/real_edge_runtime.py
Modelo detectado	Preparado	edge/models/trash_classifier.tflite

El modelo confirmado en el repositorio es TensorFlow Lite, con etiquetas por defecto `cardboard`, `glass`, `paper`, `plastic`. La salida normalizada en `ModelRunner.predict()` es clasificación:

```
[
  {
    "label": "plastic",
    "confidence": 0.91,
    "bbox": null
  }
]
```

Nota técnica. El modelo actualmente documentado es `classification-only`. No se debe presentar como `object detection` ni como sistema con `bounding boxes` reales. El campo `bbox` se publica como `null` en el runtime real porque el modelo no produce coordenadas.

Escalado conceptual a múltiples nodos:

```
flowchart LR
    subgraph Nodes["Arduino / Edge nodes"]
        n1["Arduino Node 01\nstatus metrics classification event logs"]
        n2["Arduino Node 02\nstatus metrics classification event logs"]
        n3["Arduino Node 03\nstatus metrics classification event logs"]
    end

    n1 -->|arduino/node-01/...| broker["Mosquitto"]
    n2 -->|arduino/node-02/...| broker
    n3 -->|arduino/node-03/...| broker
    broker --> backend["Backend subscriber"]
    backend --> db["PostgreSQL"]
    backend --> dashboard["Dashboard"]
```

1.6 4. MQTT Communication Layer

El root topic real es `arduino`, definido por `MQTT_TOPIC_ROOT` y usado por back-end y frontend. Los tópicos esperados son:

Tópico	Uso
<code>arduino/<device-id>/status</code>	Estado operativo del dispositivo.
<code>arduino/<device-id>/metrics</code>	Métricas de inferencia, recursos y métricas FL simuladas/telemetría.
<code>arduino/<device-id>/classification</code>	Resultado de clasificación.
<code>arduino/<device-id>/event</code>	Eventos estructurados.
<code>arduino/<device-id>/logs</code>	Logs del runtime edge.
<code>arduino/<device-id>/help</code>	Solicitudes de revisión o ayuda.

Payload real de status compatible con dashboard:

```
{
  "device_id": "unoq-01",
  "online": true,
  "status": "online",
  "state": "running",
  "mode": "simulation",
  "cpu": 55,
  "ram": 39,
  "cpu_percent": 55,
  "ram_percent": 39,
  "temp": "47.0 C",
  "heartbeat": "42 ms",
  "model_version": "simulated-v1",
  "ts": "2026-05-14T12:00:00Z"
}
```

Payload de metrics usado para cards y charts:

```
{
  "device_id": "unoq-01",
  "fps": 12.4,
  "inference_ms": 83.0,
  "cpu_percent": 55,
  "ram_percent": 39,
  "globalAccuracy": 82.4,
  "globalLoss": 0.31,
  "localLoss": 0.42,
  "localAccuracy": 80.1,
  "round": 3,
  "samplesTrained": 256,
}
```

```

    "drift": 2.1,
    "mode": "simulation",
    "ts": "2026-05-14T12:00:00Z"
}

```

Payload de clasificación:

```

{
  "device_id": "unoq-01",
  "label": "plastic",
  "confidence": 0.91,
  "bbox": null,
  "source": "real_model",
  "model_version": "trash_classifier.tflite",
  "inference_ms": 83.0,
  "ts": "2026-05-14T12:00:00Z"
}

```

Payload de evento:

```

{
  "device_id": "unoq-01",
  "type": "classification_detected",
  "severity": "info",
  "message": "plastic detected with confidence 0.91",
  "label": "plastic",
  "confidence": 0.91,
  "source": "real_model",
  "ts": "2026-05-14T12:00:00Z"
}

```

Payload de log:

```

{
  "device_id": "unoq-01",
  "level": "info",
  "message": "Inference started",
  "source": "real_model",
  "ts": "2026-05-14T12:00:00Z"
}

```

sequenceDiagram

autonumber

participant Edge as Edge simulator / real runtime

participant Broker as Mosquitto MQTT broker

participant Backend as FastAPI MQTT subscriber

participant DB as PostgreSQL

participant API as /api/dashboard/bootstrap

participant UI as Next.js dashboard

```

Edge->>Broker: PUBLISH arduino/unoq-01/status
Edge->>Broker: PUBLISH arduino/unoq-01/metrics
Edge->>Broker: PUBLISH arduino/unoq-01/classification
Broker-->>Backend: SUB arduino/+/#
Backend->>DB: INSERT mqtt_messages(topic, kind, device_id, payload_json, recorded_at)
Backend->>DB: UPSERT device_status_latest / INSERT coordinator_metrics
UI->>API: GET /api/dashboard/bootstrap
API->>DB: Read recent status, metrics, events, classifications
API-->>UI: devices, metricHistory, fl state, event streams
Broker-->>UI: MQTT WebSocket live update

```

1.7 5. Backend Architecture

El backend se implementa con FastAPI y arranca tres responsabilidades en TrashUQ/backend/app/main.py:

1. `ensure_schema()` inicializa tablas e índices.
2. `GrpcServerRuntime` expone el servicio FL gRPC.
3. `MqttIngestRuntime` se conecta a Mosquitto y se suscribe a `arduino/+/#`.

Endpoints HTTP principales:

Endpoint	Respuesta
GET /health	{"ok": true}
GET /api/dashboard/bootstrap	Estado real para dashboard: devices, metrics, metricHistory, streams y estado FL.
GET /api/fl/state	Snapshot mínimo del coordinador FL.

Componentes y puertos reales:

Component	Responsibility	Port	Technology
frontend	Dashboard, proxy API, MQTT WebSocket client	3000	Next.js
backend	REST API, MQTT ingest, gRPC FL runtime	4000, 50051	FastAPI, grpcio
mqtt	Broker MQTT y WebSocket	1883, 9001	Eclipse Mosquitto

Component	Responsibility	Port	Technology
db	Persistencia de telemetría e histórico	5432 host mapping por defecto	PostgreSQL 16
edge	Cliente, simulador, runtime real	proceso local	Python, OpenCV, MQTT, gRPC

El proxy frontend se define en `TrashUQ/frontend/app/api/[...path]/route.ts` y usa `BACKEND_API_URL=http://backend:4000` dentro de Docker. Esto evita el error clásico de intentar llamar a `127.0.0.1:4000` desde el contenedor frontend.

1.8 6. Database Layer

PostgreSQL actúa como fuente verificable del histórico MQTT. El esquema se inicializa en `TrashUQ/backend/app/db.py`.

Tablas relevantes:

Tabla	Uso
<code>mqtt_messages</code>	Registro completo de cada mensaje MQTT recibido.
<code>device_status_latest</code>	Último estado parseado por dispositivo.
<code>device_status_history</code>	Histórico de estados.
<code>coordinator_metrics</code>	Métricas agregadas derivadas de payloads <code>metrics</code> .

Campos principales de `mqtt_messages`:

Campo	Significado
<code>topic</code>	Tópico MQTT completo.
<code>kind</code>	<code>status</code> , <code>metrics</code> , <code>event</code> , <code>classification</code> , <code>help</code> , <code>logs</code> u <code>other</code> .
<code>device_id</code>	Device ID inferido del tópico.
<code>payload_text</code> / <code>payload</code>	Payload original como texto.
<code>payload_json</code>	JSONB si el payload es JSON válido.
<code>recorded_at</code>	Timestamp en milisegundos.
<code>created_at</code>	Timestamp SQL derivado.

Consulta de verificación:

```
docker compose exec db psql -U trashuq -d dashboard -c "select topic, payload, created_at fr
```

1.9 7. Frontend Dashboard

El dashboard Next.js consume datos por dos vías:

1. Bootstrap inicial y polling: `fetch("/api/dashboard/bootstrap")`.
2. Actualización live: MQTT WebSocket `ws://localhost:9001/mqtt`.

El frontend no necesita mocks para la ruta principal. Si no hay datos reales, muestra estados vacíos o mensajes de espera. Las vistas principales incluyen:

Vista / elemento	Fuente real
Active Devices	<code>devices</code> desde bootstrap + status MQTT live.
Current Round	estado FL y métricas MQTT con <code>round</code> .
Global Accuracy / Loss	<code>metrics.globalAccuracy</code> , <code>metrics.globalLoss</code> , <code>metricHistory</code> .
Avg CPU / RAM	status y metrics MQTT parseados.
Live Client Summary	estado, CPU, RAM, mode, latest classification, confidence.
Event Stream	<code>event</code> , logs y MQTT live.
Local Training Loss chart	<code>metricHistory</code> con <code>localLoss</code> / <code>globalLoss</code> .
Client Drift chart	<code>metricHistory</code> con <code>drift</code> .

Cuando no existe métrica FL real para loss o drift, el dashboard muestra waiting states como “Waiting for FL metric data...” en vez de pintar curvas falsas.

flowchart LR

```
bootstrap["GET /api/dashboard/bootstrap"] --> state["React dashboard state"]
websocket["MQTT WebSocket\narduino/+status metrics event classification help logs"] --> s

state --> cards["Cards\nActive Devices, Round, Accuracy, Training State"]
state --> summary["Live Client Summary\nstatus, CPU, RAM, mode, latest classification"]
state --> charts["Charts\nLocal/global loss, accuracy, client drift"]
state --> streams["Event Stream\nEvents, logs, help, classifications"]

db["PostgreSQL mqtt_messages\nstatus/latest metrics/history"] --> bootstrap
mqtt["Mosquitto broker"] --> websocket
```

1.10 8. Federated Learning Layer

La capa FL usa gRPC y el proto real en `TrashUQ/backend/app/fl.proto`.

RPCs disponibles:

RPC	Request	Response
Join	client_id	ok, message, round, model_version, global_weights
GetGlobalModel	client_id	ok, message, round, model_version, global_weights
SubmitUpdate	client_id, round, num_samples, local_weights, local_loss, local_accuracy	ok, message, round_aggregated, current_round, model_version

El coordinador actual mantiene estado en memoria: clientes online, ronda, versión de modelo, pesos globales y updates pendientes. El tamaño del modelo por defecto es FL_MODEL_SIZE=16 y el mínimo por ronda es FL_MIN_CLIENTS_PER_ROUND=2.

Estado actual:

Capacidad	Estado
Join desde edge	Funciona.
GetGlobalModel desde edge	Funciona.
SubmitUpdate desde edge	Se omite correctamente.
Entrenamiento local real	Pendiente.

```
sequenceDiagram
    autonumber
    participant Edge as Edge FL client
    participant FL as Backend gRPC FL coordinator :50051
    participant State as In-memory FederatedCoordinator
    participant MQTT as MQTT event/log telemetry

    Edge->>FL: Join(client_id)
    FL->>State: register online client
    State-->>FL: round, model_version, global_weights
    FL-->>Edge: JoinResponse(ok=true)

    Edge->>FL: GetGlobalModel(client_id)
    FL->>State: read current global model
    State-->>FL: round, model_version, global_weights
    FL-->>Edge: GetGlobalModelResponse(ok=true)
```

Edge->>MQTT: event fl_connected

Note over Edge,FL: SubmitUpdate is intentionally skipped today because the edge repo does

1.11 9. Edge Simulator and No-Hardware Demo

El simulador es una pieza clave para demos sin hardware. No es un mock de frontend: publica mensajes MQTT reales hacia Mosquitto, el backend los persiste en PostgreSQL y el dashboard los consume por la misma ruta que usaría un dispositivo real.

Comando principal:

```
cd ~/Documents/TrashNet/edge
uv run python -m app.edge_simulator
```

El simulador publica:

Tipo	Contenido
status	online/offline, CPU, RAM, temperatura, heartbeat, modo.
metrics	FPS, inferencia, CPU/RAM, accuracy/loss, round, samples, drift.
classification	etiquetas simuladas como trash, plastic, paper, metal, organic, clean.
event	eventos de detección.
logs	actividad del loop.
help	revisión si la confianza es baja.

1.12 10. Real Model Runtime Preparation

El repositorio edge/ ya contiene la preparación para ejecutar inferencia real:

Archivo	Función
edge/app/model_runner.py	Carga models/trash_classifier.tflite mediante el clasificador existente.
edge/app/camera_runtime.py	Fuente de frames: camera, image, video.
edge/app/real_edge_runtime.py	Loop de inferencia, MQTT publish y smoke FL no bloqueante.
edge/scripts/test_model_load.py	Verifica carga del modelo.
edge/scripts/test_camera_open.py	Verifica apertura de cámara.
edge/scripts/test_single_image_inference.py	Ejecuta inferencia sobre una imagen.

Variables principales:

```
EDGE_INPUT_SOURCE=camera
EDGE_CAMERA_INDEX=0
EDGE_IMAGE_PATH=
EDGE_VIDEO_PATH=
EDGE_MODEL_PATH=models/trash_classifier.tflite
EDGE_CONFIDENCE_THRESHOLD=0.5
EDGE_INFERENCE_INTERVAL_SEC=1
EDGE_MAX_FPS=10
```

Limitación actual. El runtime real requiere `tflite_runtime` o TensorFlow Lite compatible. En el entorno local inspeccionado, el test de modelo falla de forma explícita si esa dependencia no está instalada; esto es correcto porque no se debe simular inferencia real.

1.13 11. Verification and Testing

1.13.1 Start full stack

```
cd ~/Documents/TrashNet/TrashUQ
docker compose down
docker compose up --build
```

1.13.2 Backend health

```
curl http://localhost:4000/health
curl http://localhost:4000/api/dashboard/bootstrap
```

1.13.3 Frontend proxy

```
curl http://localhost:3000/api/dashboard/bootstrap
docker compose exec frontend sh -lc 'wget -qO- http://backend:4000/health || curl -s http://'
```

1.13.4 MQTT monitor

```
cd ~/Documents/TrashNet/TrashUQ
docker compose exec mqtt mosquitto_sub -h localhost -p 1883 -t 'arduino/+/+' -v
```

1.13.5 Edge simulator

```
cd ~/Documents/TrashNet/edge
uv run python -m app.edge_simulator
```

1.13.6 gRPC smoke

```
cd ~/Documents/TrashNet/edge
uv run python scripts/test_fl_grpc.py
```

Resultado esperado:

```
Join: ok=True ...
```

```
GetGlobalModel: ok=True ...
```

```
SubmitUpdate skipped: real local training/model update is not implemented in the edge repo y
```

1.13.7 Model load

```
cd ~/Documents/TrashNet/edge
```

```
uv run python scripts/test_model_load.py
```

1.13.8 DB verification

```
cd ~/Documents/TrashNet/TrashUQ
```

```
docker compose exec db psql -U trashuq -d dashboard -c "select topic, payload, created_at fr
```

1.14 12. Results

Resultados alcanzados:

Resultado	Estado
MQTT end-to-end desde edge a broker	Verificado.
Persistencia backend en PostgreSQL	Verificado.
Dashboard recibe datos reales por bootstrap y WebSocket	Verificado.
Simulador edge llena cards, streams y charts mediante MQTT real	Implementado.
Charts usan historial real <code>metricHistory</code> o waiting state	Implementado.
gRPC Join y <code>GetGlobalModel</code>	Verificado.
Wrapper de modelo real TFLite	Implementado/preparado.
Runtime de cámara/imagen/vídeo	Implementado/preparado.

Resultados no reclamados:

Elemento	Motivo
Entrenamiento local federado real	No hay implementación que produzca <code>local_weights</code> , <code>local_loss</code> , <code>local_accuracy</code> .
<code>SubmitUpdate</code> real	Se omite de forma explícita.

Elemento	Motivo
Bounding boxes reales	El modelo detectado es classification-only.
Ejecución real TFLite garantizada en cualquier host	Depende de instalar runtime compatible.

1.15 13. Limitaciones

- `SubmitUpdate` real no está implementado en el edge repo.
- No existe aún tracking de privacy budget.
- No existe dashboard específico de calidad de dataset.
- No se miden costes de comunicación por ronda FL.
- El modelo actual es de clasificación, no detección con bounding boxes.
- La ejecución con cámara depende del hardware, permisos del dispositivo y runtime TFLite.
- El coordinador FL mantiene estado en memoria; no persiste rondas o pesos globales en base de datos.
- La seguridad MQTT/gRPC está preparada para configuración básica, pero no hay hardening completo de producción.

1.16 14. Future Work

- Conectar cámara y hardware Arduino/edge.
- Instalar `tflite_runtime` o TensorFlow Lite en el dispositivo objetivo.
- Ejecutar stream real de clasificación y observarlo en dashboard.
- Añadir fine-tuning o entrenamiento local real en edge.
- Implementar `SubmitUpdate` con payload válido.
- Persistir rondas FL, pesos globales y métricas por cliente.
- Añadir model registry y versionado operacional.
- Implementar analytics por dispositivo.
- Añadir drift detection de dataset y modelo.
- Endurecer seguridad: autenticación MQTT, TLS, credenciales gRPC, autorización por dispositivo.
- Desplegar una flota real de nodos edge.

1.17 15. Demo Script

Guion recomendado para una demo live:

1. Arrancar la plataforma:

```
cd ~/Documents/TrashNet/TrashUQ
docker compose up --build
```

2. Abrir el dashboard:

<http://localhost:3000>

3. Ejecutar el simulador edge:

```
cd ~/Documents/TrashNet/edge
uv run python -m app.edge_simulator
```

4. Mostrar en el dashboard:

- unoq-01 online.
- CPU/RAM/heartbeat actualizándose.
- Clasificaciones en vivo.
- Event Stream con mensajes reales MQTT.
- Charts de loss/drift si llegan métricas con localLoss, globalLoss y drift.

5. Verificar persistencia:

```
cd ~/Documents/TrashNet/TrashUQ
docker compose exec db psql -U trashuq -d dashboard -c "select topic, payload, created_at fr
```

6. Verificar gRPC:

```
cd ~/Documents/TrashNet/edge
uv run python scripts/test_fl_grpc.py
```

7. Explicar readiness de cámara/modelo:

```
cd ~/Documents/TrashNet/edge
EDGE_CAMERA_INDEX=0 uv run python scripts/test_camera_open.py
uv run python scripts/test_model_load.py
```

1.18 16. Deployment Topology

flowchart TB

```
subgraph Host["Developer / demo host"]
  subgraph Compose["Docker Compose: TrashUQ"]
    frontend["frontend\nNext.js\n0.0.0.0:3000 -> 3000"]
    backend["backend\nFastAPI + gRPC\n0.0.0.0:4000 -> 4000\n0.0.0.0:50051 -> 50051"]
    mqtt["mqtt\nEclipse Mosquitto\n0.0.0.0:1883 -> 1883\n0.0.0.0:9001 -> 9001"]
    db["db\nPostgreSQL 16\n0.0.0.0:5432 -> 5432"]
  end
  edgeLocal["edge repo\nuv run python -m app.edge_simulator\nor app.real_edge_runtime"]
end
```

```
external["External edge device\nMQTT_HOST=<SERVER_LAN_IP>\nFL_GRPC_HOST=<SERVER_LAN_IP>"]
browser["Browser\nhttp://localhost:3000"]
```

```
browser --> frontend
frontend --> |internal HTTP| backend
frontend --> |browser WebSocket| mqtt
backend --> db
```



```

backend --> mqtt
edgeLocal --> mqtt
edgeLocal --> backend
external --> mqtt
external --> backend

```

1.19 17. Glossary

Term	Definición
Edge AI	Inferencia de IA ejecutada cerca del sensor/dispositivo.
MQTT	Protocolo publish/subscribe ligero para telemetría.
gRPC	RPC binario basado en HTTP/2 y Protocol Buffers.
Federated Learning	Entrenamiento distribuido donde los clientes envían updates, no datos crudos.
Global model	Modelo agregado mantenido por el coordinador FL.
Local update	Actualización entrenada localmente por un cliente edge.
Dashboard bootstrap	Estado inicial servido por backend para reconstruir el dashboard.
Telemetry	Métricas, eventos, logs y estado operacional enviados por dispositivos.
TFLite	TensorFlow Lite, runtime/model format optimizado para edge.

1.20 18. Conclusión

TrashUQ ya demuestra una canalización real edge-to-cloud: un cliente edge publica MQTT, Mosquitto enruta mensajes, FastAPI persiste en PostgreSQL, y el dashboard visualiza estado y métricas en vivo. El simulador permite una demo completa sin hardware, pero no falsifica datos en frontend: todo atraviesa la infraestructura real. La integración gRPC valida el camino de coordinación federada con `Join` y `GetGlobalModel`, y el runtime TFLite prepara el salto hacia cámara/modelo real. El siguiente hito técnico es cerrar el bucle de entrenamiento local: generar updates válidos en edge, enviar `SubmitUpdate` y persistir métricas FL de producción por ronda y dispositivo.